

```
1 package com.astromedicomp.win_rtr4;
2
3 import android.content.Context; // for drawing context related
4 import android.opengl.GLSurfaceView; // for OpenGL Surface View and all related
5 import javax.microedition.khronos.opengles.GL10; // for OpenGL ES 1.0 needed as param type GL10
6 import javax.microedition.khronos.egl.EGLConfig; // for EGLConfig needed as param type EGLConfig
7 import android.opengl.GLES32; // for OpenGL ES 3.2
8 import android.view.MotionEvent; // for "MotionEvent"
9 import android.view.GestureDetector; // for GestureDetector
10 import android.view.GestureDetector.OnGestureListener; // OnGestureListener
11 import android.view.GestureDetector.OnDoubleTapListener; // for OnDoubleTapListener
12
13 import java.nio.ByteBuffer;
14 import java.nio.ByteOrder;
15 import java.nio.FloatBuffer;
16
17 import android.opengl.Matrix; // for Matrix math
18
19 // A view for OpenGL ES3 graphics which also receives touch events
20 public class GLESView extends GLSurfaceView implements GLSurfaceView.Renderer, OnGestureListener, OnDoubleTapListener
21 {
22     private final Context context;
23
24     private GestureDetector gestureDetector;
25
26     private int vertexShaderObject;
27     private int fragmentShaderObject;
28     private int shaderProgramObject;
29
30     private int[] vao = new int[1];
31     private int[] vbo = new int[1];
32     private int mvpUniform;
33
34     private float orthographicProjectionMatrix[]=new float[16]; // 4x4 matrix
35
36     public GLESView(Context drawingContext)
37     {
38         super(drawingContext);
39
40         context=drawingContext;
41
42         // accordingly set EGLContext to current supported version of OpenGL-ES
43         setEGLContextClientVersion(3);
44
45         // set Renderer for drawing on the GLSurfaceView
46         setRenderer(this);
47
48         // Render the view only when there is a change in the drawing data
49         setRenderMode(GLSurfaceView.RENDERMODE_WHEN_DIRTY);
50
```

```
51     gestureDetector = new GestureDetector(context, this, null, false);
52     gestureDetector.setOnDoubleTapListener(this);
53 }
54
55 // overridden method of GLSurfaceView.Renderer ( Init Code )
56 @Override
57 public void onSurfaceCreated(GL10 gl, EGLConfig config)
58 {
59     // get OpenGL-ES version
60     String glesVersion = gl.glGetString(GL10.GL_VERSION);
61     System.out.println("VDG: OpenGL-ES Version = "+glesVersion);
62     // get GLSL version
63     String glslVersion=gl.glGetString(GLES32.GL_SHADING_LANGUAGE_VERSION);
64     System.out.println("VDG: GLSL Version = "+glslVersion);
65
66     initialize(gl);
67 }
68
69 // overridden method of GLSurfaceView.Renderer ( Chnge Size Code )
70 @Override
71 public void onSurfaceChanged(GL10 unused, int width, int height)
72 {
73     resize(width, height);
74 }
75
76 // overridden method of GLSurfaceView.Renderer ( Rendering Code )
77 @Override
78 public void onDrawFrame(GL10 unused)
79 {
80     display();
81 }
82
83 // Handling 'onTouchEvent' Is The Most IMPORTANT,
84 // Because It Triggers All Gesture And Tap Events
85 @Override
86 public boolean onTouchEvent(MotionEvent e)
87 {
88     // code
89     int eventaction = e.getAction();
90     if(!gestureDetector.onTouchEvent(e))
91         super.onTouchEvent(e);
92     return(true);
93 }
94
95 // abstract method from OnDoubleTapListener so must be implemented
96 @Override
97 public boolean onDoubleTap(MotionEvent e)
98 {
99     return(true);
100 }
101
102 // abstract method from OnDoubleTapListener so must be implemented
103 @Override
104 public boolean onDoubleTapEvent(MotionEvent e)
105 {
106     // Do Not Write Any code Here Because Already Written 'onDoubleTap'
```

```
107         return(true);
108     }
109
110     // abstract method from OnDoubleTapListener so must be implemented
111     @Override
112     public boolean onSingleTapConfirmed(MotionEvent e)
113     {
114         return(true);
115     }
116
117     // abstract method from OnGestureListener so must be implemented
118     @Override
119     public boolean onDown(MotionEvent e)
120     {
121         // Do Not Write Any code Here Because Already Written
122         'onSingleTapConfirmed'
123         return(true);
124     }
125
126     // abstract method from OnGestureListener so must be implemented
127     @Override
128     public boolean onFling(MotionEvent e1, MotionEvent e2, float velocityX,
129                           float velocityY)
130     {
131         return(true);
132     }
133
134     // abstract method from OnGestureListener so must be implemented
135     @Override
136     public void onLongPress(MotionEvent e)
137     {
138     }
139
140     // abstract method from OnGestureListener so must be implemented
141     @Override
142     public boolean onScroll(MotionEvent e1, MotionEvent e2, float distanceX,
143                           float distanceY)
144     {
145         uninitialized();
146         System.exit(0);
147         return(true);
148     }
149
150     // abstract method from OnGestureListener so must be implemented
151     @Override
152     public void onShowPress(MotionEvent e)
153     {
154     }
155
156     // abstract method from OnGestureListener so must be implemented
157     @Override
158     public boolean onSingleTapUp(MotionEvent e)
159     {
160         return(true);
161     }
162 }
```

```
160 private void initialize(GL10 gl)
161 {
162     // *****
163     // Vertex Shader
164     // *****
165     // create shader
166     vertexShaderObject=GL ES32.glCreateShader(GL ES32.GL_VERTEX_SHADER);
167
168     // vertex shader source code
169     final String vertexShaderSourceCode =String.format
170     (
171         "#version 320 es"+
172         "\n"+
173         "in vec4 vPosition;" +
174         "uniform mat4 u_mvp_matrix;" +
175         "void main(void)" +
176         "{" +
177         "gl_Position = u_mvp_matrix * vPosition;" +
178         "}"
179     );
180
181     // provide source code to shader
182     GL ES32.glShaderSource(vertexShaderObject,vertexShaderSourceCode);
183
184     // compile shader & check for errors
185     GL ES32.glCompileShader(vertexShaderObject);
186     int[] iShaderCompiledStatus = new int[1];
187     int[] iInfoLogLength = new int[1];
188     String szInfoLog=null;
189     GL ES32.glGetShaderiv(vertexShaderObject, GL ES32.GL_COMPILE_STATUS,
190         iShaderCompiledStatus, 0);
191     if (iShaderCompiledStatus[0] == GL ES32.GL_FALSE)
192     {
193         GL ES32.glGetShaderiv(vertexShaderObject,
194             GL ES32.GL_INFO_LOG_LENGTH, iInfoLogLength, 0);
195         if (iInfoLogLength[0] > 0)
196         {
197             szInfoLog = GL ES32.glGetShaderInfoLog(vertexShaderObject);
198             System.out.println("VDG: Vertex Shader Compilation Log =
199                 "+szInfoLog);
200             uninitialize();
201             System.exit(0);
202         }
203     }
204
205     // *****
206     // Fragment Shader
207     // *****
208     // create shader
209     fragmentShaderObject=GL ES32.glCreateShader(GL ES32.GL_FRAGMENT_SHADER);
210
211     // fragment shader source code
212     final String fragmentShaderSourceCode =String.format
    (
        "#version 320 es"+
        "\n"+
```

```
213         "precision highp float;"+
214         "out vec4 FragColor;"+
215         "void main(void)"+
216         "{"+
217         "FragColor = vec4(1.0, 1.0, 1.0, 1.0);"+
218         "}"
219     );
220
221     // provide source code to shader
222     GLES32.glShaderSource(fragmentShaderObject, fragmentShaderSourceCode);
223
224     // compile shader and check for errors
225     GLES32.glCompileShader(fragmentShaderObject);
226     iShaderCompiledStatus[0] = 0; // re-initialize
227     iInfoLogLength[0] = 0; // re-initialize
228     szInfoLog=null; // re-initialize
229     GLES32.glGetShaderiv(fragmentShaderObject, GLES32.GL_COMPILE_STATUS,
230                          iShaderCompiledStatus, 0);
231     if (iShaderCompiledStatus[0] == GLES32.GL_FALSE)
232     {
233         GLES32.glGetShaderiv(fragmentShaderObject,
234                             GLES32.GL_INFO_LOG_LENGTH, iInfoLogLength, 0);
235         if (iInfoLogLength[0] > 0)
236         {
237             szInfoLog = GLES32.glGetShaderInfoLog(fragmentShaderObject);
238             System.out.println("VDG: Fragment Shader Compilation Log = "+szInfoLog);
239             uninitialize();
240             System.exit(0);
241         }
242     }
243
244     // create shader program
245     shaderProgramObject=GLES32.glCreateProgram();
246
247     // attach vertex shader to shader program
248     GLES32.glAttachShader(shaderProgramObject, vertexShaderObject);
249
250     // attach fragment shader to shader program
251     GLES32.glAttachShader(shaderProgramObject, fragmentShaderObject);
252
253     // pre-link binding of shader program object with vertex shader
254     // attributes
255     GLES32.glBindAttribLocation
256         (shaderProgramObject, GLSMacros.VDG_ATTRIBUTE_VERTEX, "vPosition");
257
258     // link the two shaders together to shader program object
259     GLES32.glLinkProgram(shaderProgramObject);
260     int[] iShaderProgramLinkStatus = new int[1];
261     iInfoLogLength[0] = 0; // re-initialize
262     szInfoLog=null; // re-initialize
263     GLES32.glGetProgramiv(shaderProgramObject, GLES32.GL_LINK_STATUS,
264                          iShaderProgramLinkStatus, 0);
265     if (iShaderProgramLinkStatus[0] == GLES32.GL_FALSE)
266     {
267         GLES32.glGetProgramiv(shaderProgramObject,
```

```
        GLES32.GL_INFO_LOG_LENGTH, iInfoLogLength, 0);
263     if (iInfoLogLength[0] > 0)
264     {
265         szInfoLog = GLES32.glGetProgramInfoLog(shaderProgramObject);
266         System.out.println("VDG: Shader Program Link Log = " + szInfoLog);
267         uninitialized();
268         System.exit(0);
269     }
270 }
271
272 // get MVP uniform location
273 mvpUniform=GLES32.glGetUniformLocation(shaderProgramObject,"u_mvp_matrix");
274
275 // *** vertices, colors, shader attribs, vbo, vao initializations ***
276 final float triangleVertices[]= new float[]
277 {
278     0.0f, 50.0f,0.0f, // apex
279     -50.0f,-50.0f,0.0f, // left-bottom
280     50.0f,-50.0f,0.0f // right-bottom
281 };
282
283 GLES32.glGenVertexArrays(1,vao,0);
284 GLES32.glBindVertexArray(vao[0]);
285
286 GLES32.glGenBuffers(1,vbo,0);
287 GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER,vbo[0]);
288
289 ByteBuffer byteBuffer=ByteBuffer.allocateDirect
    (triangleVertices.length * 4);
290 byteBuffer.order(ByteOrder.nativeOrder());
291 FloatBuffer verticesBuffer=byteBuffer.asFloatBuffer();
292 verticesBuffer.put(triangleVertices);
293 verticesBuffer.position(0);
294
295 GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER,
296                     triangleVertices.length * 4,
297                     verticesBuffer,
298                     GLES32.GL_STATIC_DRAW);
299
300 GLES32.glVertexAttribPointer(GLES32.GL_VERTEX_ATTRIB_VERTEX,
301                             3,
302                             GLES32.GL_FLOAT,
303                             false,0,0);
304
305 GLES32.glEnableVertexAttribArray(GLES32.GL_VERTEX_ATTRIB_VERTEX);
306
307 GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER,0);
308 GLES32.glBindVertexArray(0);
309
310 // enable depth testing
311 GLES32.glEnable(GLES32.GL_DEPTH_TEST);
312 // depth test to do
313 GLES32.glDepthFunc(GLES32.GL_EQUAL);
314 // We will always cull back faces for better performance
```

```
315     GLES32.glEnable(GLES32.GL_CULL_FACE);
316
317     // Set the background color
318     GLES32.glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
319
320     // set projectionMatrix to identity matrix
321     Matrix.setIdentityM(orthographicProjectionMatrix,0);
322 }
323
324 private void resize(int width, int height)
325 {
326     // code
327     GLES32.glViewport(0, 0, width, height);
328
329     // Orthographic Projection => left,right,bottom,top,near,far
330     if (width <= height)
331         Matrix.orthoM(orthographicProjectionMatrix,0,-100.0f, 100.0f,
332             (-100.0f * (height / width)), (100.0f * (height / width)),
333             -100.0f, 100.0f);
334     else
335         Matrix.orthoM(orthographicProjectionMatrix,0,(-100.0f * (width /
336             height)), (100.0f * (width / height)), -100.0f, 100.0f, -100.0f,
337             100.0f);
338 }
339
340 public void display()
341 {
342     // code
343     GLES32.glClear(GLES32.GL_COLOR_BUFFER_BIT |
344         GLES32.GL_DEPTH_BUFFER_BIT);
345
346     // use shader program
347     GLES32.glUseProgram(shaderProgramObject);
348
349     // OpenGL-ES drawing
350     float modelViewMatrix[]=new float[16];
351     float modelViewProjectionMatrix[]=new float[16];
352
353     // set modelview & modelviewprojection matrices to identity
354     Matrix.setIdentityM(modelViewMatrix,0);
355     Matrix.setIdentityM(modelViewProjectionMatrix,0);
356
357     // multiply the modelview and projection matrix to get
358     // modelviewprojection matrix
359     Matrix.multiplyMM
360         (modelViewProjectionMatrix,0,orthographicProjectionMatrix,0,modelVie
361         wMatrix,0);
362
363     // pass above modelviewprojection matrix to the vertex shader in
364     // 'u_mvp_matrix' shader variable
365     // whose position value we already calculated in initWithFrame() by
366     // using glGetUniformLocation()
367     GLES32.glUniformMatrix4fv
368         (mvpUniform,1,false,modelViewProjectionMatrix,0);
369
370     // bind vao
```

```
360     GLES32.glBindVertexArray(vao[0]);
361
362     // draw, either by glDrawTriangles() or glDrawArrays() or ?
363     GLES32.glDrawArrays(GLES32.GL_TRIANGLES,0,3); // 3 (each with its ?
364     x,y,z ) vertices in triangleVertices array
365
366     // unbind vao
367     GLES32.glBindVertexArray(0);
368
369     // un-use shader program
370     GLES32.glUseProgram(0);
371
372     // render/flush
373     requestRender();
374 }
375
376 void uninitialize()
377 {
378     // code
379     // destroy vao
380     if(vao[0] != 0)
381     {
382         GLES32.glDeleteVertexArrays(1, vao, 0);
383         vao[0]=0;
384     }
385
386     // destroy vbo
387     if(vbo[0] != 0)
388     {
389         GLES32.glDeleteBuffers(1, vbo, 0);
390         vbo[0]=0;
391     }
392
393     if(shaderProgramObject != 0)
394     {
395         if(vertexShaderObject != 0)
396         {
397             // detach vertex shader from shader program object ?
398             GLES32.glDetachShader(shaderProgramObject,
399             vertexShaderObject);
400             // delete vertex shader object
401             GLES32.glDeleteShader(vertexShaderObject);
402             vertexShaderObject = 0;
403         }
404
405         if(fragmentShaderObject != 0)
406         {
407             // detach fragment shader from shader program object ?
408             GLES32.glDetachShader(shaderProgramObject,
409             fragmentShaderObject);
410             // delete fragment shader object
411             GLES32.glDeleteShader(fragmentShaderObject);
412             fragmentShaderObject = 0;
413         }
414     }
415 }
```

```
412
413     // delete shader program object
414     if(shaderProgramObject != 0)
415     {
416         GLES32.glDeleteProgram(shaderProgramObject);
417         shaderProgramObject = 0;
418     }
419 }
420 }
421
```